

RAG (RETRIEVAL AUGMENTED GENERATION) - STUDY NOTES

Source: YouTube Video Transcript by Nitish

WHAT IS RAG?

RAG = Retrieval Augmented Generation

A way to make an LLM smarter by giving it EXTRA RELEVANT INFORMATION at the time you ask a question – without retraining the model.

RAG is a marriage of two concepts:

- Information Retrieval (old Computer Science topic)
- Text Generation (via LLMs)

THE BIG STORY (EASY LANGUAGE)

LLMs like ChatGPT are trained on massive internet data and store all that knowledge in their "parameters" (weights and biases). When you ask a question, they pull from this parametric knowledge.

But there are 3 situations where this fails...

WHY RAG WAS NEEDED – 3 PROBLEMS WITH LLMs

PROBLEM 1: PRIVATE DATA

- LLMs never saw your company/personal data during training
- So they simply cannot answer questions about it
- Example: Asking ChatGPT about a specific internal company video or document – it has no idea

PROBLEM 2: KNOWLEDGE CUTOFF DATE

- Every LLM has a training cutoff date
- After that date, it has NO knowledge of new events
- Example: Asking about today's news → LLM can't answer
- (ChatGPT can answer recent things because it has web search enabled – open source models do not)

PROBLEM 3: HALLUCINATION

- LLMs sometimes give CONFIDENTLY WRONG answers
- Because text generation is probabilistic, the model can "imagine" a plausible-sounding but completely false answer

49 - Example: Model says "Einstein played football for Germany"
50 - totally made up, but said with confidence
51
52 -----
53 FIRST SOLUTION TRIED: FINE-TUNING
54 -----
55 What is Fine-tuning?
56 Take a pre-trained LLM and retrain it on a smaller,
57 domain-specific dataset.
58
59 Analogy:
60 - Student completing engineering degree = Pre-training (LLM)
61 - Company giving 2-3 months job training = Fine-tuning
62
63 Types of Fine-tuning:
64 1. Supervised Fine-tuning (SFT)
65 - Provide labeled data: Prompt → Desired Output pairs
66 - Feed 1,000 to 1,000,000 such pairs
67 - Most popular method
68
69 2. Continued Pre-training
70 - Unsupervised – just feed raw domain text
71 - Same process as original pre-training but on smaller data
72
73 3. RLHF (Reinforcement Learning from Human Feedback)
74 - Combines reinforcement learning + human feedback
75 - Teaches model how to behave in real scenarios
76
77 4. LoRA / QLoRA
78 - Parameter efficient fine-tuning methods
79 - Freeze base weights, train only remaining weights
80 - Much cheaper than full fine-tuning
81
82 Fine-tuning Steps (Supervised):
83 Step 1: Collect labeled domain data (Prompt + Desired Output)
84 Step 2: Choose method (Full fine-tuning or LoRA/QLoRA)
85 Step 3: Train the model for some epochs
86 Step 4: Evaluate (exact match, factuality, hallucination rate)
87
88 Problems with Fine-tuning:
89 ✗ Computationally very expensive
90 ✗ Requires strong technical expertise (AI engineers)
91 ✗ Must retrain every time data is updated
92 ✗ Hard to remove outdated information
93 ✗ Not practical for frequently changing data
94
95 -----
96 BETTER CONCEPT: IN-CONTEXT LEARNING (ICL)

97 -----
98 Definition:
99 A core capability of large LLMs where the model learns to
100 solve a task purely by seeing examples in the prompt –
101 WITHOUT updating its weights.
102
103 Key Point – Emergent Property:
104 ICL is an EMERGENT PROPERTY of LLMs.
105 - Not explicitly programmed
106 - Appeared naturally as models got bigger
107 - GPT-1, GPT-2 → did NOT have reliable ICL
108 - GPT-3 (175 billion parameters) → ICL appeared automatically
109
110 Landmark Paper:
111 "Language Models Are Few-Shot Learners"
112 - First formal paper on in-context learning
113 - Showed GPT-3 can learn tasks from examples in the prompt
114
115 Example of ICL (Few-Shot Prompting):
116 Tell model: "I love this phone" → Positive
117 "This app crashes a lot" → Negative
118 "The camera is amazing" → Positive
119 Ask model: "I hate the battery life" → ?
120 Model answers: Negative ✓
121
122 Types of prompting in ICL:
123 - Zero-shot: No examples, just ask directly
124 - Few-shot: Give a few examples, then ask
125
126 ICL Limitation:
127 Not guaranteed to give good results on every task type.
128 Later models (GPT-3.5, GPT-4) improved ICL significantly
129 through alignment techniques (SFT + RLHF).
130
131 -----
132 THE RAG CONCEPT – ICL TAKEN FURTHER
133 -----
134 Instead of giving a few EXAMPLES in the prompt (few-shot),
135 RAG injects the actual CONTEXT needed to answer the question.
136
137 Simple Example:
138 - Website has 2-hour video lecture on Linear Regression
139 - Student asks a doubt about Gradient Descent
140 - RAG sends the transcript of ONLY the Gradient Descent
141 section (e.g., minute 5 to 25) along with the question
142 - LLM reads that context and answers the doubt
143
144 This is called Context Injection.

```
145
146 RAG Definition:
147     "RAG is a way to make a language model smarter by giving it
148     extra information at the time you ask your question."
149
150 -----
151 THE RAG PIPELINE – 4 STEPS IN DETAIL
152 -----
153
154 STEP 1: INDEXING (Build the External Knowledge Base)
155 -----
156 Definition: Process of preparing your knowledge base so it
157 can be efficiently searched at query time.
158
159 Sub-steps inside Indexing:
160
161     1a. DOCUMENT INGESTION
162         - Load source data into memory
163         - Sources: PDFs, YouTube, websites, AWS S3, Google Drive
164         - Tools: LangChain Document Loaders
165             (PDF Loader, YouTube Loader, Web Base Loader, etc.)
166
167     1b. TEXT CHUNKING
168         - Break large documents into smaller, meaningful chunks
169         - Why? Two reasons:
170             (i) LLM context window has token limits
171             (ii) Semantic search works better on smaller chunks
172         - Chunks should be semantically meaningful (no abrupt cuts)
173         - Tools: LangChain Text Splitters
174             - Recursive Character Text Splitter (most popular)
175             - Semantic Chunker
176             - HTML-based splitter, Markdown-based splitter, etc.
177
178     1c. EMBEDDING GENERATION
179         - Convert each text chunk → dense vector (embedding)
180         - Captures the MEANING of the text mathematically
181         - Uses an Embedding Model:
182             - OpenAI Embeddings
183             - Sentence Transformers
184             - Other embedding models
185         - Result: Each chunk now has a corresponding vector
186
187     1d. VECTOR STORE (Save Everything)
188         - Store vectors + original text + metadata together
189         - This becomes your External Knowledge Base
190         - Local options: FAISS, Chroma
191         - Cloud options: Pinecone, Weaviate, Milvus, Qdrant
192
```

```
193 STEP 2: RETRIEVAL (Find Relevant Context)
194 -----
195 Definition: Real-time process of finding most relevant pieces
196 of information from the vector store based on user's query.
197
198 Sub-steps inside Retrieval:
199
200 2a. Query Embedding
201     - Convert user's question → embedding vector
202     - Use the SAME embedding model used in indexing
203
204 2b. Semantic Search
205     - Search vector store for closest vectors to query vector
206     - Search types: Simple semantic search, MMR,
207       Contextual Compression, advanced reranking
208
209 2c. Ranking
210     - Order results by relevance
211     - Metrics: Cosine Similarity, or advanced reranking algos
212
213 2d. Fetch Context
214     - Retrieve original text chunks of top 3-5 results
215     - These chunks become your CONTEXT
216
217 STEP 3: AUGMENTATION (Build the Final Prompt)
218 -----
219 - Combine user's query + retrieved context into one prompt
220 - Prompt structure example:
221     "You are a helpful assistant. Answer the question ONLY
222     from the provided context. If the context is insufficient,
223     say you don't know."
224     [CONTEXT: ...transcript text here...]
225     [QUESTION: How does gradient descent work?]
226
227 - This is called CONTEXT INJECTION
228 - You are augmenting the LLM's parametric knowledge with
229   external knowledge
230
231 STEP 4: GENERATION (LLM Produces the Answer)
232 -----
233 - Send the augmented prompt to the LLM
234 - LLM combines its parametric knowledge + injected context
235 - Generates a final, grounded response for the user
236
237 -----
238 HOW RAG SOLVES THE 3 ORIGINAL PROBLEMS
239 -----
240
```

241 PROBLEM 1 SOLVED – Private Data:

242 The external knowledge base is built from YOUR OWN data.

243 Context comes from your data → answers come from your data.

244  LLM can now answer private data questions

246 PROBLEM 2 SOLVED – Knowledge Cutoff:

247 Add new documents/articles to vector store anytime.

248 NO retraining needed.

249 Just generate new embeddings and add to the store.

250  LLM stays up to date cheaply and easily

252 PROBLEM 3 SOLVED – Hallucination:

253 You explicitly instruct the model: "Answer only from context."

254 "If context is insufficient, say I don't know."

255 This GROUNDS the model's response to real information.

256  Hallucination chances are greatly reduced

258 -----

259 FINE-TUNING vs RAG – COMPARISON TABLE

260 -----

Aspect	Fine-tuning	RAG
Cost	Very expensive	Much cheaper
Model retraining	Required	Not required
Data updates	Retrain each time	Just update vector DB
Technical expertise	High (AI engineers)	Moderate
Hallucination fix	Partial	Better (grounded)
Private data	Bakes into model	External knowledge base
Speed of update	Slow	Fast
Complexity	High	Lower

273 -----

274 INTERVIEW-CRITICAL TERMS TO KNOW

275 -----

Term	Meaning
Parametric knowledge	Knowledge stored in LLM's weights
In-context learning (ICL)	LLM learns from prompt examples
Few-shot prompting	Giving examples in the prompt
Emergent property	Ability that appears at scale
Embeddings / Dense vectors	Numerical vector representation of text
Semantic search	Search by meaning, not keyword
Vector store / Vector DB	Database to store and search vectors
Chunking	Breaking large text into small pieces
Context injection	Adding relevant info into the prompt
Cosine similarity	Metric to measure vector closeness

289	MMR retrieval	Maximal Marginal Relevance (diverse results)
290	Fine-tuning	Retraining LLM on domain-specific data
291	Supervised fine-tuning	Fine-tuning with labeled Q&A pairs
292	Continued pre-training	Unsupervised domain-specific training
293	LoRA / QLoRA	Parameter-efficient fine-tuning methods
294	RLHF	Reinforcement Learning from Human Feedback
295	Hallucination	LLM confidently giving wrong answers
296	Knowledge cutoff	Date after which LLM has no information
297	Grounding	Anchoring LLM response to a given context

298
299 -----

300 TOOLS & FRAMEWORKS MENTIONED

301 -----

302
303 Framework: LangChain

- 304 - Document Loaders (PDF, YouTube, Web, S3, etc.)
- 305 - Text Splitters (Recursive, Semantic, HTML, Markdown)
- 306 - Vector Store integrations
- 307 - Retriever components

308
309 Vector Stores:

- 310 Local: FAISS, Chroma
- 311 Cloud: Pinecone, Weaviate, Milvus, Qdrant

312
313 Embedding Models:

- 314 - OpenAI Embeddings
- 315 - Sentence Transformers
- 316 - Any other embedding model

317
318 LLMs compatible with RAG:

- 319 - GPT-3, GPT-3.5, GPT-4 (OpenAI)
- 320 - Claude (Anthropic)
- 321 - LLaMA (Meta)
- 322 - Any open-source LLM

323
324 -----

325 KEY TAKEAWAY FOR INTERVIEWS

326 -----

327
328 RAG is preferred over Fine-tuning when:

- 329 ✓ Your data changes frequently
- 330 ✓ You have private/domain-specific documents
- 331 ✓ You need up-to-date information
- 332 ✓ You want to reduce hallucinations
- 333 ✓ You have budget/compute constraints

334
335 Fine-tuning is preferred when:

- 336 ✓ You want to change the model's BEHAVIOR or STYLE

337

338

339

340

341

342

343

344

✓

You need to teach the model a new language or format

✓

The task requires deeply embedded domain knowledge

✓

Data is stable and doesn't change often

=====

END OF NOTES

=====

LANGCHAIN DOCUMENT LOADERS – INTERVIEW STUDY NOTES

Based on LangChain RAG Playlist

1. WHAT ARE DOCUMENT LOADERS?

Definition:

Document Loaders are LangChain components that fetch data from any external source and convert it into a standardized "Document object" format, ready for use in a RAG pipeline (chunking, embedding, retrieval, generation).

Where to find them:

All document loaders live in the `langchain_community.document_loaders` package. They are contributed by the open-source community, not the core LangChain team. That is why they are in the "community" package.

Usage pattern (same for ALL loaders):

Step 1: Create loader object (pass file path or URL)
Step 2: Call `.load()` on the object
Step 3: Get back a Python list of Document objects

Example:

```
from langchain_community.document_loaders import TextLoader
loader = TextLoader("my_file.txt", encoding="utf-8")
docs = loader.load()
```

2. THE DOCUMENT OBJECT – STANDARDIZED FORMAT

Every Document object has exactly TWO fields:

[1] `page_content` → The actual text content of the data
[2] `metadata` → Extra information about the document

What metadata typically includes:

- `source` (file path or URL)
- `page` (page number, for PDFs)
- `total_pages` (total pages in PDF)
- `author` (document author)
- `creation_date` (when file was created)
- `title` (document title)
- `row` (row number, for CSVs)

```
49
50 How to access them:
51 docs[0].page_content    # get the text
52 docs[0].metadata        # get the metadata dict
53
54
55
```

56 3. THE 4 MOST IMPORTANT DOCUMENT LOADERS

59 LOADER 1: TextLoader

```
62 Purpose      : Load plain text files (.txt)
63 Use cases    : Log files, code snippets, transcripts, subtitles
64 Output       : 1 Document object per file (entire file = 1 doc)
65 Parameter    : file path + optional encoding (e.g. "utf-8")
66 Key note     : Simplest loader in LangChain
67
```

68 Code:

```
69 from langchain_community.document_loaders import TextLoader
70 loader = TextLoader("cricket.txt", encoding="utf-8")
71 docs = loader.load()
72 print(len(docs))          # → 1
73 print(docs[0].page_content)
74 print(docs[0].metadata)
75
```

77 LOADER 2: PyPDFLoader

```
79 Purpose      : Load PDF files, page by page
80 Use cases    : Books, research papers, company documents
81 Output       : 1 Document object per PAGE (25-page PDF = 25 docs)
82 Parameter    : file path
83 Dependency   : pip install pypdf
84 Key note     : Works on page-by-page basis (important!)
85
```

86 When to use PyPDFLoader:

- 87 - PDF contains mostly text content
- 88 - Simple layout without heavy formatting

90 When NOT to use PyPDFLoader:

- 91 - Scanned image PDFs → use UnstructuredPDFLoader
- 92 - PDFs with tables → use PDFPlumberLoader
- 93 - Complex layouts → use PyMuPDFLoader

95 Code:

```
96 from langchain_community.document_loaders import PyPDFLoader
```

```
97 loader = PyPDFLoader("document.pdf")
98 docs = loader.load()
99 print(len(docs))          # → number of pages
100 print(docs[0].page_content) # → text from page 1
101 print(docs[0].metadata)    # → source, page, total_pages, etc.
```

103

104 **LOADER 3: WebBaseLoader**

105

106 Purpose : Load and extract text from web pages

107 Use cases : Blogs, news articles, product pages, documentation

108 Output : 1 Document object per URL

109 Parameter : single URL or list of URLs

110 Libraries : Uses Requests (HTTP) + BeautifulSoup (HTML parsing)

111

112 Limitation : Works best with static HTML pages.

113 For JavaScript-heavy pages → use SeleniumURLLoader

114

115 Code:

```
116 from langchain_community.document_loaders import WebBaseLoader
117 loader = WebBaseLoader("https://example.com/article")
118 docs = loader.load()
119 print(len(docs))          # → 1 (for single URL)
120 print(docs[0].page_content) # → extracted text
```

121

122 Multiple URLs:

```
123 loader = WebBaseLoader(["https://url1.com", "https://url2.com"])
124 docs = loader.load()
125 print(len(docs))          # → 2 (one doc per URL)
```

126

127

128 **LOADER 4: CSVLoader**

129

130 Purpose : Load CSV files for data Q&A

131 Use cases : Sales data, survey results, analytics, datasets

132 Output : 1 Document object per ROW (400 rows = 400 docs)

133 Parameter : file path

134

135 page_content format: "column1: value1\ncolumn2: value2\n..."

136 metadata includes : source filename, row number

137

138 Code:

```
139 from langchain_community.document_loaders import CSVLoader
140 loader = CSVLoader("data.csv")
141 docs = loader.load()
142 print(len(docs))          # → number of rows in CSV
143 print(docs[0])            # → first row as document
144 print(docs[1])            # → second row as document
```

145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192

4. DIRECTORYLOADER – BATCH LOADING MULTIPLE FILES

Purpose: Load all files from a folder at once using a single loader.
Works with any other document loader (not just PDF).

- Three required parameters:
- 1. path → directory/folder path
 - 2. glob → file pattern to match (which files to load)
 - 3. loader_cls → which document loader class to use

Glob pattern examples:

- "*.pdf" → all PDF files in the root directory
- "**/*.txt" → all .txt files in all subfolders
- "data/*.csv" → all CSV files inside the "data" folder
- "**/*" → all files in all subfolders

Code:

```
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader

loader = DirectoryLoader(
    path="books/",
    glob="*.pdf",
    loader_cls=PyPDFLoader
)

docs = loader.load()
```

Real example from video:

3 PDFs with 326 + 392 + 468 pages = 1186 total document objects
(DirectoryLoader combines all pages from all PDFs into one list)

Access specific pages:

- docs[0] # first page of first PDF
- docs[325] # last page of first PDF (0-indexed)
- docs[326] # first page of second PDF

5. LOAD() VS LAZY_LOAD() – CRITICAL INTERVIEW CONCEPT

Both functions are available on EVERY document loader in LangChain.

Feature	load()	lazy_load()

193			
194	Loading style	Eager loading	Lazy loading (on demand)
195	Returns	Python list of Documents	Python generator of Documents
196	Memory usage	Loads ALL docs into RAM	Loads ONE doc at a time
197	Output timing	Wait for all → then output	Output starts immediately
198	Best for	Small/few files	Large/many files
199	Risk	Memory overflow with big data	Slightly slower per-item
200			

201

202 When to use load():

- 203 - Working with 1-2 files
- 204 - Small documents
- 205 - Need all content in memory at once

206

207 When to use lazy_load():

- 208 - Large CSV files (millions of rows)
- 209 - Hundreds of PDFs
- 210 - Stream processing without loading everything into RAM
- 211 - Memory-constrained environments

212

213 Code comparison:

```
214 # Eager loading
215 docs = loader.load()
216 for doc in docs:      # iterate over list
217     print(doc.metadata)
218
219 # Lazy loading
220 docs = loader.lazy_load()
221 for doc in docs:      # iterate over generator (one at a time)
222     print(doc.metadata)
```

223

224

226 6. PDF LOADER VARIANTS – PICK THE RIGHT ONE

229	Loader	Best For
230		
231	PyPDFLoader	Text-based PDFs with simple layout (most common)
232	PDFPlumberLoader	PDFs with tables and structured tabular data
233	UnstructuredPDFLoader	Scanned image PDFs, OCR, complex structure
234	PyMuPDFLoader	PDFs with rich formatting, images, complex layout
235	AmazonTextractPDFLoader	Scanned images via AWS Textract OCR service

239 7. OTHER IMPORTANT LOADER CATEGORIES

240

Cloud Storage:

- S3FileLoader / S3DirectoryLoader (Amazon S3)
- AzureBlobStorageFileLoader (Microsoft Azure)
- GoogleDriveLoader (Google Drive)
- DropboxLoader (Dropbox)

Web Scraping:

- WebBaseLoader (static HTML – blogs, news)
- SeleniumURLLoader (JavaScript-heavy dynamic pages)
- UnstructuredURLLoader

Video & Audio:

- YoutubeLoader (YouTube video transcripts)

Productivity Tools:

- NotionDBLoader (Notion pages/databases)
- ConfluenceLoader (Confluence pages)
- GitLoader (Git repositories)

Messaging & Communication:

- SlackDirectoryLoader (Slack export)
- DiscordChatLoader (Discord export)

File Types:

- JSONLoader (JSON files with jq schema)
- CSVLoader (CSV files)
- DirectoryLoader (any folder of files)
- UnstructuredFileLoader (catch-all for many formats)

Key advice: Do NOT memorize all loaders. Learn the concept.

Look up the specific loader from docs when you need it in a project.

8. CUSTOM DOCUMENT LOADERS

If no built-in loader exists for your data source, create your own.

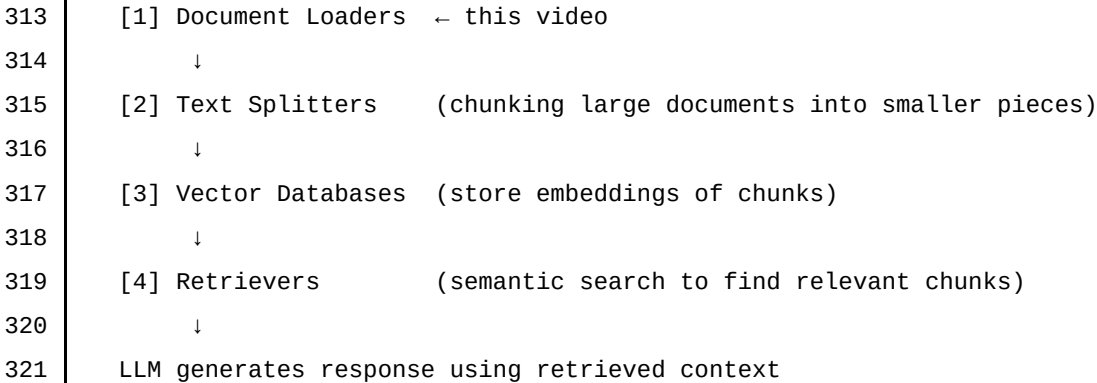
How:

1. Create a Python class
2. Inherit from BaseLoader (LangChain's base class)
3. Implement .load() and/or .lazy_load() methods
4. Return Document objects from those methods

This is how all 100+ community loaders were originally created.

```
289 Basic template:
290 from langchain_core.document_loaders import BaseLoader
291 from langchain_core.documents import Document
292
293 class MyCustomLoader(BaseLoader):
294     def __init__(self, source):
295         self.source = source
296
297     def lazy_load(self):
298         # your custom logic to fetch data
299         # yield one Document at a time
300         yield Document(
301             page_content="your text here",
302             metadata={"source": self.source}
303         )
304
305     def load(self):
306         return list(self.lazy_load())
307
308
309
```

310 9. THE 4 RAG COMPONENTS – BIG PICTURE



325 10. INTERVIEW GLOSSARY – KEY TERMS

326

327

Term	Definition
Document object	Standardized LangChain format with page_content + metadata fields
page_content	The actual text content of a loaded document
metadata	Dictionary of extra info (source, page, author, etc.)
Document Loader	LangChain component that loads data from a source and converts it to Document objects
langchain_community	Package where all community-built loaders live

330

331

332

333

334

335

336

337	TextLoader	Loads plain .txt files
338	PyPDFLoader	Loads PDFs page by page (1 page = 1 Document)
339	WebBaseLoader	Loads web pages using Requests + BeautifulSoup
340	CSVLoader	Loads CSV files row by row (1 row = 1 Document)
341	DirectoryLoader	Loads all matching files from a folder at once
342	glob pattern	File matching pattern (e.g. *.pdf, **/*.txt)
343	loader_cls	Parameter in DirectoryLoader specifying which
344		loader class to use for each file
345	Eager loading	load() – loads all documents into RAM at once
346	Lazy loading	lazy_load() – loads one document at a time via
347		a Python generator
348	Generator	Python object that yields values one at a time
349		instead of returning all at once (memory-efficient)
350	BeautifulSoup	Python library for parsing HTML structure
351	BaseLoader	LangChain base class for creating custom loaders
352	Custom loader	User-defined document loader inheriting BaseLoader
353	PDFPlumberLoader	PDF loader specialized for tables
354	UnstructuredPDFLoader	PDF loader for scanned images / OCR
355	PyMuPDFLoader	PDF loader for complex layouts and rich formatting
356	SeleniumURLLoader	Web loader for JavaScript-heavy dynamic pages
357	YoutubeLoader	Loads YouTube video transcripts

358
359

360
361 11. QUICK RECALL CHEAT SHEET

362		
363		
364	All loaders return	→ list of Document objects
365	Document has	→ page_content + metadata
366	PyPDFLoader output	→ 1 document per PAGE
367	CSVLoader output	→ 1 document per ROW
368	WebBaseLoader output	→ 1 document per URL
369	DirectoryLoader needs	→ path + glob + loader_cls
370	load() returns	→ Python list (all in memory at once)
371	lazy_load() returns	→ Python generator (one at a time)
372	Use load() when	→ small number of files
373	Use lazy_load() when	→ large number of files
374	WebBaseLoader uses	→ Requests + BeautifulSoup
375	For JS-heavy pages use	→ SeleniumURLLoader
376	For scanned PDFs use	→ UnstructuredPDFLoader
377	For PDFs with tables use	→ PDFPlumberLoader
378	All loaders found in	→ langchain_community.document_loaders
379	Custom loader inherits from	→ BaseLoader

380
381
382
383
384

12. EASY LANGUAGE SUMMARY

385

386 This video teaches the FIRST building block of any RAG application –

387 Document Loaders.

388

389 THE CORE IDEA:

390 To build a RAG app, you need data. That data can live anywhere – in a PDF,

391 a text file, a website, a CSV, or cloud storage. Document Loaders are

392 LangChain's way of picking up data from any of these places and converting

393 it into a standard format that the rest of the RAG pipeline understands.

394

395 THE STANDARD FORMAT:

396 No matter where the data comes from, you always get back Document objects.

397 Each has two things: the actual text (page_content) and info about the

398 document (metadata like filename, page number, etc.)

399

400 THE FOUR MAIN LOADERS:

401 - TextLoader : for .txt files

402 - PyPDFLoader : for PDFs (one document per page)

403 - WebBaseLoader : for websites (one document per URL)

404 - CSVLoader : for spreadsheets (one document per row)

405

406 THE TRICKY PART (load vs lazy_load):

407 - load() dumps everything into RAM at once – fine for small files

408 - lazy_load() processes one item at a time – use for big files

409 Think of it like: load() = reading the whole book at once,

410 lazy_load() = reading one page, processing it, then moving to the next.

411

412 DIRECTORYLOADER:

413 Instead of loading files one by one, DirectoryLoader lets you point to a

414 whole folder and load everything inside that matches a pattern.

415 3 PDFs with 326 + 392 + 468 pages = 1186 Document objects automatically.

416

417 CUSTOM LOADERS:

418 If LangChain doesn't have a loader for your data source, you can build your

419 own by inheriting from BaseLoader. This is exactly how the 100+ community

420 loaders were built.

421

422 BIG PICTURE:

423 Document Loaders → Text Splitters → Vector DB → Retrievers → LLM

424 Master all four components and you can build any RAG application.

425

426 =====

427 Generated from LangChain Document Loaders Tutorial – Study Notes

428 =====

429

TEXT SPLITTING IN LangChain - STUDY NOTES

Source: YouTube Video Transcript by Nitish

WHAT IS TEXT SPLITTING?

Text splitting is the process of breaking large chunks of text (articles, PDFs, books, code files) into smaller, manageable pieces that an LLM can handle effectively.

The code/tool that performs this operation is called a TEXT SPLITTER.

Golden Rule in LLM world:

NEVER try to deal with very large text all at once.

Always split first → then process.

Splitting greatly improves LLM output quality.

WHY IS TEXT SPLITTING IMPORTANT? (3 REASONS)

REASON 1: OVERCOMES CONTEXT LENGTH LIMITS

- Every LLM has a maximum input size (context window)
- Example: If LLM allows 50,000 tokens but your PDF has 100,000 words → you CANNOT send it directly
- Text splitting solves this by breaking it into smaller parts
- Same applies to embedding models (they also have input limits)

REASON 2: IMPROVES QUALITY OF DOWNSTREAM TASKS

a) Better Embedding Quality:

- If you embed a large text as one chunk, the vector cannot capture all the semantic meaning properly
- Smaller, focused chunks produce better, more accurate vectors
- Example: A doc about CSK + RCB + MI – embedding all together is bad; separate chunks for each team = better

b) Better Semantic Search:

- Semantic search works much better on smaller chunks
- The retrieved results are more precise and relevant

c) Better Summarization:

- LLMs "drift" on very large texts (start saying things not in the document)
- Smaller chunks give more accurate, focused summaries

REASON 3: OPTIMIZES COMPUTATIONAL RESOURCES

- Less memory needed to process smaller text

- Enables parallel processing (process multiple chunks at once)
- Faster and cheaper overall

4 TYPES OF TEXT SPLITTING TECHNIQUES

TYPE 1: LENGTH-BASED TEXT SPLITTING

What it is:

Simplest and fastest method. You decide a fixed chunk size (in characters or tokens) and split at that exact point.

How it works:

- Set chunk size = 100 characters
- Start reading from the beginning
- At every 100 characters → create a new chunk
- Resume from where you stopped → repeat

LangChain Class: CharacterTextSplitter

Code Example:

```
from langchain.text_splitter import CharacterTextSplitter

splitter = CharacterTextSplitter(
    chunk_size=100,      # size of each chunk
    chunk_overlap=0,      # overlap between chunks
    separator=""          # no separator = pure character split
)

chunks = splitter.split_text(your_text)
# OR for document objects:
chunks = splitter.split_documents(docs)
```

Advantages:

- ✓ Very simple to understand
- ✓ Easy to implement
- ✓ Very fast

Disadvantages:

- ✗ Does NOT consider linguistic structure or grammar
- ✗ Can cut in the MIDDLE of a word or sentence
- ✗ Split context causes poor embedding quality
- ✗ Not used much in practice

IMPORTANT CONCEPT: CHUNK OVERLAP

What is chunk overlap?

The number of characters/tokens that are SHARED between two consecutive chunks.

103 Example (chunk_size=100, chunk_overlap=20):
104 Chunk 1: characters 1 to 100
105 Chunk 2: characters 81 to 180 ← starts 20 chars BEFORE chunk 1 ends
106 (characters 81-100 appear in BOTH chunks)

107
108 Why use chunk overlap?
109 - When text is cut abruptly, you lose context at the boundary
110 - Overlap ensures the context from the end of chunk 1 is
111 also present at the start of chunk 2
112 - Helps preserve meaning across chunk boundaries

113
114 Trade-off:
115 More overlap = better context preservation BUT more chunks
116 → more computation and cost

117
118 Recommended overlap:
119 For text-based applications: 10% to 20% of chunk size
120 If chunk_size = 100 → chunk_overlap = 10 to 20
121 If chunk_size = 1000 → chunk_overlap = 100 to 200

122 -----
123
124 TYPE 2: TEXT STRUCTURE-BASED SPLITTING
125 -----

126 What it is:
127 Uses the natural hierarchical structure of text to decide
128 WHERE to split (not just HOW MANY characters).

129
130 Core Insight:
131 All text has a natural hierarchy:
132 Paragraphs → Sentences → Words → Characters

133
134 LangChain Class: RecursiveCharacterTextSplitter
135 (THE MOST COMMONLY USED TEXT SPLITTER)

136
137 Default separators used (in order):
138 1. "\n\n" → paragraph break (try this FIRST)
139 2. "\n" → line/sentence break
140 3. " " → space (word level)
141 4. "" → character level (last resort)

142
143 How the algorithm works (step by step):
144 1. Try to split on paragraphs (\n\n)
145 → If all resulting chunks ≤ chunk_size → DONE
146 → If any chunk > chunk_size → go to step 2
147
148 2. Try to split on sentences (\n)
149 → If all chunks ≤ chunk_size → DONE
150 → If any chunk > chunk_size → go to step 3
151
152 3. Try to split on words (spaces)
153 → If all chunks ≤ chunk_size → DONE

→ If any chunk > chunk_size → go to step 4

4. Split on characters (last resort)

OPTIMIZATION (Merging small chunks):

After splitting, the algorithm also tries to MERGE small adjacent chunks together (without exceeding chunk_size).

This reduces the total number of chunks and keeps more context together.

Effect of chunk_size on splitting level:

chunk_size very small (e.g., 10) → splits at word level

chunk_size medium (e.g., 25) → splits at sentence level

chunk_size large (e.g., 50+) → splits at paragraph level

chunk_size very large → entire text = 1 chunk

LangChain Code:

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=300,
```

```
    chunk_overlap=30
```

```
)
```

```
chunks = splitter.split_text(your_text)
```

Advantages over CharacterTextSplitter:

✓ Respects natural text boundaries (paragraphs, sentences)

✓ Words are NOT cut in the middle

✓ More semantically meaningful chunks

✓ Most widely used in real RAG applications

Disadvantages:

✗ Slightly more complex

✗ Still doesn't understand semantic meaning

TYPE 3: DOCUMENT STRUCTURE-BASED SPLITTING

What it is:

An EXTENSION of recursive character text splitting for special document types (NOT plain English text).

When to use:

When your document is organized by programming constructs or markup language syntax, NOT by paragraphs/sentences.

Examples:

- Python code (organized by class, def, loops)

- JavaScript code

- HTML files

- Markdown files

205

206 How it works:

207 Same RecursiveCharacterTextSplitter algorithm, BUT with

208 DIFFERENT SEPARATORS specific to the document type.

209

210 For Python code, separators include:

211 "\n\n" → splits at class definition

212 "\n\n" → splits at function definition

213 Then falls back to normal text separators

214

215 LangChain Code (Python):

```
216 splitter = RecursiveCharacterTextSplitter.from_language(
217     language="python", # specify language here
218     chunk_size=300,
219     chunk_overlap=30
220 )
221 chunks = splitter.split_text(python_code_text)
```

222

223 Supported languages in LangChain:

224 python, javascript, java, php, html, markdown, and more

225

226 For Markdown:

```
227 splitter = RecursiveCharacterTextSplitter.from_language(
228     language="markdown",
229     chunk_size=175,
230     chunk_overlap=0
231 )
```

232

233 Key point:

234 The algorithm is the SAME as Type 2 – only the separators

235 change to match the document's structure.

236

237 -----

238 TYPE 4: SEMANTIC MEANING-BASED SPLITTING

239 -----

240 What it is:

241 Uses EMBEDDING MODELS to understand the MEANING of text

242 and splits where the meaning changes significantly.

243

244 Problem it solves:

245 Types 1, 2, and 3 all fail when a SINGLE PARAGRAPH contains

246 TWO COMPLETELY DIFFERENT TOPICS mixed together.

247 Example: One paragraph discusses farming AND IPL cricket.

248 Structure-based splitting keeps them in one chunk.

249 Semantic splitting separates them properly.

250

251 How it works (algorithm):

252 Step 1: Break text into individual sentences

253 Step 2: Generate embedding vector for each sentence

254 (using OpenAI or any embedding model)

255 Step 3: Calculate cosine similarity between every pair

```
256         of consecutive sentences:
257         sim(S1, S2), sim(S2, S3), sim(S3, S4)...
258     Step 4: Find where similarity DROPS SUDDENLY
259         → That drop = topic change = splitting point
260
261 LangChain Class: SemanticChunker
262 (from langchain_experimental – still experimental!)
263
264 LangChain Code:
265     from langchain_experimental.text_splitter import SemanticChunker
266     from langchain_openai.embeddings import OpenAIEmbeddings
267
268     splitter = SemanticChunker(
269         embeddings=OpenAIEmbeddings(),
270         breakpoint_threshold_type="standard_deviation",
271         breakpoint_threshold_amount=1 # 1 std deviation
272     )
273     chunks = splitter.split_text(text)
274
275 Threshold types available:
276     - standard_deviation (most common)
277     - percentile
278     - interquartile
279     - gradient
280
281 How threshold works:
282     threshold_amount=1 → only split if similarity drops by
283                         MORE than 1 standard deviation
284     threshold_amount=3 → requires a BIGGER drop to split
285                         (more tolerant, fewer chunks)
286
287 Current status:
288     ⚠ Still experimental in LangChain
289     ⚠ Results can be inconsistent
290     ✅ Conceptually the most intelligent approach
291     ✅ Will improve as embedding models get better
292     ✅ Promising for the future
293
294 -----
295 COMPARISON OF ALL 4 TECHNIQUES
296 -----
297
298 Technique          | Simplicity | Quality | Use in Practice
299 -----|-----|-----|-----
300 1. Length (Character) | Very Easy | Low     | Rarely used
301 2. Structure (RCTS)  | Medium    | High    | MOST USED ✅
302 3. Document-based    | Medium    | High    | Common for code
303 4. Semantic          | Complex   | Highest | Experimental
304
305 RCTS = RecursiveCharacterTextSplitter
306
```

307 RECOMMENDATION:
308 Use RecursiveCharacterTextSplitter for most RAG applications.
309 Use from_language() variant for code and markdown files.
310 Use SemanticChunker only when topic boundaries are unclear.

311 -----
312
313 HOW TO CONNECT DOCUMENT LOADER + TEXT SPLITTER
314 -----

315 (The standard workflow in any RAG pipeline)

316

```
317 from langchain_community.document_loaders import PyPDFLoader
318 from langchain.text_splitter import RecursiveCharacterTextSplitter
319
320 # Step 1: Load the document
321 loader = PyPDFLoader("your_file.pdf")
322 docs = loader.load()
323
324 # Step 2: Split into chunks
325 splitter = RecursiveCharacterTextSplitter(
326     chunk_size=500,
327     chunk_overlap=50
328 )
329 chunks = splitter.split_documents(docs)
330
331 # Each chunk is a Document object with:
332 # chunk.page_content → the actual text
333 # chunk.metadata    → source, page number, etc.

334



335 Key functions:



```
336 split_text(text_string) → for plain strings
337 split_documents(list_of_docs) → for Document objects
```


```

338 -----
339
340 INTERVIEW-CRITICAL TERMS TO KNOW
341 -----

342		
343	Term	Meaning
344	-----	-----
345	Text splitting / Chunking	Breaking large text into smaller pieces
346	Chunk size	Max size of each chunk (chars/tokens)
347	Chunk overlap	Shared characters between adjacent chunks
348	CharacterTextSplitter	Split every N characters (simplest)
349	RecursiveCharacterTextSplitter	Split respecting text structure (BEST)
350	from_language()	RCTS variant for code/markdown
351	SemanticChunker	Splits based on embedding similarity
352	Semantic search	Search by meaning using vectors
353	Cosine similarity	Metric to compare how similar two vectors are
354	Standard deviation threshold	How much similarity change = split point
355	Context window	Max tokens an LLM can accept at once
356	Downstream tasks	Tasks done AFTER text splitting (embedding,
357		search, summarization, generation)

358 Sliding window approach | Comparing each sentence with the next one
359 Breakpoint | Where the semantic chunker decides to split
360

361 -----

362 QUICK REFERENCE: CHUNK SIZE GUIDELINES

363 -----

365 Use case	Chunk Size	Overlap
366 -----	-----	-----
367 General RAG / Q&A	500-1000	50-100
368 Code splitting (Python, JS)	300-500	30-50
369 Markdown documents	150-300	20-50
370 Very detailed/technical docs	200-500	50-100
371 Short documents / QA pairs	100-200	10-20

372
373 Rule of thumb for overlap:
374 Overlap = 10-20% of chunk_size
375

376 -----

377 KEY TAKEAWAYS FOR INTERVIEWS

378 -----

- 379
- 380 1. TEXT SPLITTING IS MANDATORY in LLM applications – never
381 send huge documents directly to LLMs or embedding models.
382
 - 383 2. THREE REASONS to split:
384 (a) Context length limits of LLMs/embedding models
385 (b) Better quality for embeddings, search, summarization
386 (c) Lower compute cost + enables parallelization
387
 - 388 3. BEST SPLITTER for most cases = RecursiveCharacterTextSplitter
389 because it respects paragraph → sentence → word hierarchy.
390
 - 391 4. CHUNK OVERLAP is important to avoid losing context at
392 chunk boundaries. Keep it 10-20% of chunk size.
393
 - 394 5. FOR CODE AND MARKDOWN use RecursiveCharacterTextSplitter
395 .from_language() with the appropriate language setting.
396
 - 397 6. SEMANTIC CHUNKING is the most intelligent approach but
398 still experimental – understand the concept even if you
399 don't use it yet.
400
 - 401 7. CHUNK SIZE affects splitting level:
402 Small size → word-level splitting
403 Medium size → sentence-level splitting
404 Large size → paragraph-level splitting
405
 - 406 8. Each chunk returned by split_documents() is a Document
407 object with page_content and metadata attributes.
408

409	=====
410	END OF NOTES
411	=====
412	

VECTOR STORES IN LangChain - STUDY NOTES

Source: YouTube Video by Nitesh

SIMPLE SUMMARY (Easy Language)

This video explains Vector Stores – a special type of database that stores text as numbers (called vectors) and lets you find similar content by MEANING, not just by matching words.

The teacher uses a MOVIE RECOMMENDATION example:

- Keyword matching fails because two movies can have the same director/actor but completely different stories.
- The better approach is to convert movie plots into vectors and find movies with similar vector positions.
- That is called SEMANTIC SEARCH.

Vector stores solve 3 problems:

1. STORING those vectors efficiently
2. SEARCHING them by similarity
3. Doing it FAST using indexing

The video then shows how to use ChromaDB with LangChain to create a vector store, add documents, search, update, and delete – all with a few simple function calls.

WHY VECTOR STORES EXIST – THE CORE PROBLEM

PROBLEM: Keyword Matching Fails at Meaning

Example 1 (False Positive):

Movies "My Name Is Khan" and "Kabhi Alvida Na Kehna" share:

- Same director (Karan Johar)
- Same actor (Shah Rukh Khan)
- Same genre (Drama)

So keyword system calls them SIMILAR.

But their stories are completely different – BAD recommendation.

Example 2 (False Negative):

"Taare Zameen Par" and "A Beautiful Mind" have very similar stories (a person battling illness while being brilliant).

But keywords are completely different – different director, different actors, different era.

So keyword system NEVER marks them as similar – BAD result.

KEY INSIGHT:

52 Keyword matching looks at surface metadata.
53 Semantic search looks at MEANING.
54 Vector stores enable semantic search.
55
56 -----
57 3 CHALLENGES THAT VECTOR STORES SOLVE
58 -----
59
60 CHALLENGE 1: GENERATING EMBEDDINGS
61 - You have millions of movie plots (text).
62 - You must convert each plot into a numerical vector using an
63 embedding model (like OpenAI Embeddings).
64 - This is computationally heavy but done once.
65
66 CHALLENGE 2: STORAGE
67 - You cannot store vectors in regular SQL databases like
68 MySQL or Oracle.
69 - Reason: relational databases do not support similarity
70 calculations on vectors.
71 - You need a DIFFERENT type of database → Vector Store.
72
73 CHALLENGE 3: SEMANTIC SEARCH (Speed Problem)
74 - To find the most similar movie out of 1 million movies,
75 you must compare your query vector with all 1 million vectors.
76 - That is LINEAR search = $O(n)$ = 1 million comparisons.
77 - Very slow → bad user experience.
78 - Need a SMARTER approach = Indexing.
79
80 -----
81 WHAT IS A VECTOR STORE – DEFINITION
82 -----
83
84 Official Definition:
85 "A vector store is a system designed to store and retrieve
86 data represented as numerical vectors."
87
88 Simple meaning:
89 If your application works with vectors (storing them,
90 searching them), you use a vector store.
91 It performs much better than traditional relational databases
92 for this job.
93
94 -----
95 4 CORE FEATURES OF EVERY VECTOR STORE
96 -----
97
98 FEATURE 1: STORAGE
99 - Option A: In-memory (RAM) → fast but data lost on restart
100 - Option B: On-disk → persistent, data survives restart
101 - Also stores METADATA alongside vectors (extra info like
102 movie ID, movie name, player's IPL team, etc.)

- Use in-memory for small/prototype apps.

- Use on-disk for production apps.

FEATURE 2: SIMILARITY SEARCH

- Compare a query vector with all stored vectors.

- Returns vectors most similar to the query.

- Uses Cosine Similarity to calculate how similar two vectors are (based on angle between them).

- Smaller angle = more similar.

FEATURE 3: INDEXING (for fast similarity search)

- Organizes vectors smartly so you don't compare all of them.

- Makes similarity search much faster.

- (See next section for how it works.)

FEATURE 4: CRUD OPERATIONS

- Create: add new vectors

- Read: retrieve vectors

- Update: update existing vectors

- Delete: delete outdated vectors

Same as any normal database.

HOW INDEXING WORKS – SIMPLIFIED EXPLANATION

The Problem:

1 million vectors → 1 million comparisons per query → too slow.

Clustering-Based Indexing Solution (step by step):

Step 1: Group all 1 million vectors into 10 clusters using a clustering algorithm.

Step 2: Calculate the CENTROID of each cluster.

Centroid = average of all vectors in that cluster.

Now you have 10 centroid vectors.

Step 3: When a new query vector comes in:

→ Compare query with only the 10 centroids.

→ Find which centroid is closest to the query.

→ That tells you which cluster to search in.

Step 4: Search only within that 1 cluster (~1 lakh vectors) instead of all 10 lakh vectors.

Result:

Old way: 10,00,000 comparisons

New way: 10 (centroids) + 1,00,000 (one cluster) = ~1,00,010

That is roughly 10x fewer comparisons!

154 This technique is called:
155 → APPROXIMATE NEAREST NEIGHBOR (ANN) lookup
156 → The result is "approximate" but good enough in practice.
157
158 -----
159 VECTOR STORE vs VECTOR DATABASE – KEY DIFFERENCE
160 -----
161
162 VECTOR STORE:
163 - A lightweight library or service.
164 - Does ONLY: storage + similarity search.
165 - Does NOT have: transactions, auth, backup/restore,
166 distributed architecture, concurrency control.
167 - Good for: prototyping and small-scale apps.
168 - Famous example: FAISS (by Facebook/Meta)
169
170 VECTOR DATABASE:
171 - A full-fledged database system.
172 - Does: storage + similarity search + extra DB features.
173 - Has:
174 * Distributed architecture (scale to any size)
175 * Backup and restore (if data lost, recover it)
176 * ACID transactions
177 * Concurrency control (multiple users at same time)
178 * Authentication and authorization (security)
179 - Good for: production environments and large datasets.
180 - Famous examples: Pinecone, Qdrant, Milvus, Weaviate
181
182 THE GOLDEN RULE:
183 Every vector database IS a vector store.
184 But NOT every vector store is a vector database.
185 (Because not every vector store has database-like features.)
186
187 Formula: Vector Database = Vector Store + Database features
188
189 -----
190 LANGCHAIN'S APPROACH TO VECTOR STORES
191 -----
192
193 WHY LANGCHAIN HAS VECTOR STORE SUPPORT:
194 LangChain creators understood early that LLM applications
195 would work heavily with embedding vectors.
196 So LangChain has built-in wrappers for all major vector
197 stores from the start.
198
199 COMMON INTERFACE PHILOSOPHY:
200 LangChain wraps all vector stores with the SAME method names.
201 This means:
202 - similarity_search() works the same in Chroma, Pinecone,
203 Qdrant, Weaviate, FAISS
204 - If you switch from ChromaDB to Pinecone tomorrow,

you only change the import and init line.

- The rest of your code stays the same.

Key common methods in LangChain for all vector stores:

```
from_documents()          → create vector store
add_documents() / add_text() → add new vectors
similarity_search()       → semantic search
update_document()         → update a vector
delete()                  → delete a vector
```

CHROMADB – DEEP DIVE

WHAT IS CHROMADB:

- Lightweight, open-source vector database.
- Best for: local development + small-to-medium production.
- Sits between a vector store and a full vector database.
- It HAS some database features (persistence, collections) but is lighter than Pinecone etc.
- Internally stores data as a SQLite3 file on disk.
(You can open it in any SQL client if needed.)

CHROMADB DATA HIERARCHY:

```
Tenant (user/org/team)
├── Database
│   ├── Collection (like a table in RDBMS)
│   │   ├── Document
│   │   │   ├── Embedding vector
│   │   │   └── Metadata
```

CHROMADB WITH LANGCHAIN – CODE EXAMPLES

INSTALL REQUIRED LIBRARIES:

```
pip install langchain chromadb langchain-openai
```

IMPORTS:

```
from langchain_openai import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.schema import Document
```

STEP 1 – Create Document objects:

```
doc1 = Document(
    page_content="Virat Kohli is one of the most successful batsmen in IPL history.",
    metadata={"team": "RCB"}
)
doc2 = Document(
    page_content="Rohit Sharma is the most successful captain in IPL history.",
    metadata={"team": "MI"}
```

```

256 )
257 doc3 = Document(
258     page_content="MS Dhoni is known for his calm leadership and finishing ability.",
259     metadata={"team": "CSK"}
260 )
261 doc4 = Document(
262     page_content="Jasprit Bumrah is one of the best fast bowlers in T20 cricket.",
263     metadata={"team": "MI"}
264 )
265 doc5 = Document(
266     page_content="Ravindra Jadeja is a great all-rounder who can bat, bowl, and field.",
267     metadata={"team": "CSK"}
268 )
269 docs = [doc1, doc2, doc3, doc4, doc5]
270
271 STEP 2 – Create vector store:
272 vector_store = Chroma.from_documents(
273     documents=docs,
274     embedding=OpenAIEmbeddings(),
275     persist_directory="./chroma_db",    # save on disk
276     collection_name="samples"          # like a table name
277 )
278 # When this runs:
279 # → docs are converted to embedding vectors automatically
280 # → vectors are stored in chroma_db folder as SQLite3 file
281 # → each document gets a unique auto-generated ID
282
283 STEP 3 – Add more documents later:
284 new_doc = Document(page_content="Hardik Pandya is an explosive all-rounder.", metadata={"team": "MI"})
285 ids = vector_store.add_documents([new_doc])
286 # You can also pass your own IDs if needed
287
288 STEP 4 – View all documents:
289 result = vector_store.get(include=["embeddings", "documents", "metadatas"])
290 # result contains:
291 # result["ids"] → list of IDs
292 # result["documents"] → list of text content
293 # result["embeddings"] → list of embedding vectors
294 # result["metadatas"] → list of metadata dicts
295
296 STEP 5 – Similarity search (no score):
297 results = vector_store.similarity_search(
298     query="who is a bowler?",
299     k=1    # return top 1 most similar result
300 )
301 # returns a list of Document objects
302 # k=1 → returns only Jasprit Bumrah
303 # k=2 → returns Bumrah + Jadeja (all-rounder also bowls)
304
305 STEP 6 – Similarity search with score:

```



```

306     results = vector_store.similarity_search_with_score(
307         query="who is a bowler?",
308         k=2
309     )
310     # returns list of (Document, score) tuples
311     # IMPORTANT: lower score = more similar
312     # (score represents DISTANCE, not percentage similarity)
313
314 STEP 7 – Filter by metadata:
315     results = vector_store.similarity_search(
316         query="",
317         filter={"team": "CSK"},
318         k=5
319     )
320     # returns only documents where metadata team = CSK
321     # Result: MS Dhoni + Ravindra Jadeja
322
323 STEP 8 – Update a document:
324     updated_doc = Document(
325         page_content="Virat Kohli, former captain of RCB, is renowned for his aggressive leadership.",
326         metadata={"team": "RCB"}
327     )
328     vector_store.update_document(
329         document_id="<paste-virat-kohli-doc-id-here>",
330         document=updated_doc
331     )
332
333 STEP 9 – Delete a document:
334     vector_store.delete(ids=["<paste-doc-id-here>"])
335     # After this, that document is permanently removed
336
337 -----
338 EMBEDDINGS – QUICK RECAP (How They Work)
339 -----
340
341 What is an embedding?
342     - A technique to represent the MEANING of any text as numbers.
343     - Text → Neural Network → Vector (list of numbers)
344     - Example: a 512-dimensional vector = list of 512 numbers.
345     - Similar meaning texts → similar vectors (close in space).
346
347 How it solves the movie problem:
348     1. Take the plot of every movie.
349     2. Pass it through an embedding model → get a vector per movie.
350     3. Plot all vectors in a coordinate system (512-D space).
351     4. Movies with similar stories → their vectors point in similar
352        directions (small angular distance).
353     5. Use cosine similarity to measure that angle.
354     6. Small angle = high similarity = good recommendation.
355
356 -----

```

357 USE CASES OF VECTOR STORES
358 -----
359
360 1. Recommender Systems
361 (movies, products, articles – find similar items by content)
362
363 2. Semantic Search
364 (search by meaning, not just keywords)
365
366 3. RAG Applications (Retrieval Augmented Generation)
367 (the main use case being built in this LangChain playlist)
368
369 4. Image and Multimedia Search
370 (images can also be converted to vectors and searched)
371
372 -----
373 INTERVIEW HIGHLIGHTS – DO NOT MISS THESE
374 -----
375
376 Q1: What is a vector store?
377 A: A system designed to store and retrieve data represented as
378 numerical vectors.
379
380 Q2: Why can't we use MySQL/PostgreSQL for storing vectors?
381 A: Relational databases don't support similarity calculations
382 on vectors. They're built for exact matches and range
383 queries, not vector distance math.
384
385 Q3: What is semantic search?
386 A: Search by meaning using embedding vectors, rather than
387 matching keywords. It finds results that are conceptually
388 related even if they use different words.
389
390 Q4: What is cosine similarity?
391 A: A metric to compare how similar two vectors are by measuring
392 the angle between them. Smaller angle = more similar.
393
394 Q5: What is the difference between a vector store and a vector database?
395 A: A vector store handles only storage + similarity search.
396 A vector database adds: distributed architecture, backup/restore,
397 transactions, concurrency control, and authentication.
398 Every vector database IS a vector store, but not vice versa.
399
400 Q6: How does indexing speed up vector search?
401 A: Instead of comparing a query with all N vectors (slow),
402 indexing organizes vectors into clusters. You first compare
403 with cluster centroids (only a few), find the best cluster,
404 then search within that cluster. This is Approximate Nearest
405 Neighbor (ANN) lookup – much faster.
406
407 Q7: What is a centroid?

408 A: The average vector of all vectors in a cluster. Used as the
409 "representative" of the cluster during indexed search.
410
411 Q8: What is metadata in a vector store?
412 A: Extra information stored alongside the embedding vector.
413 For example: movie ID, team name, author, page number.
414 Used for filtering search results.
415
416 Q9: What does 'k' mean in similarity_search()?
417 A: The number of top similar results to return.
418 k=1 → only the most similar result.
419 k=5 → top 5 most similar results.
420
421 Q10: What does the score mean in similarity_search_with_score()?
422 A: The score represents DISTANCE between vectors.
423 LOWER score = MORE similar (not the other way around).
424
425 Q11: Where does ChromaDB store data?
426 A: As a SQLite3 file on disk (when using persist_directory).
427 Can be opened in any SQL client if needed.
428
429 Q12: Why does LangChain use a common interface for all vector stores?
430 A: So you can swap one vector store for another with minimal
431 code changes. All major vector stores share the same method
432 names in LangChain.
433
434 Q13: What are the 4 core features of any vector store?
435 A: Storage (in-memory or on-disk), Similarity search,
436 Indexing (for fast search), CRUD operations.
437
438 Q14: Name some popular vector databases.
439 A: Pinecone, Qdrant, Milvus, Weaviate, ChromaDB.
440 FAISS (by Facebook) is a famous vector store (not full DB).
441
442 Q15: What is FAISS?
443 A: Facebook AI Similarity Search – a lightweight vector store
444 library from Meta. Not a full vector database.
445

446 -----

447 CHROMADB HIERARCHY – SUMMARY TABLE

448 -----

449

Level	Equivalent In RDBMS	Notes
-----	-----	-----
Tenant	User / Org	Top level
Database	Database	Tenant can have many DBs
Collection	Table	DB can have many collections
Document	Row	Has embedding + metadata
Embedding	Column (special)	The numerical vector
Metadata	Other columns	Extra info about the doc

458

```
459 -----
460 KEY FUNCTIONS QUICK REFERENCE
461 -----
462
463 Function | What it does
464 -----|-----
465 Chroma.from_documents() | Create vector store
466 vector_store.add_documents() | Add new documents
467 vector_store.get() | View all stored data
468 vector_store.similarity_search() | Search by meaning (no score)
469 similarity_search_with_score() | Search + get distance score
470 vector_store.update_document() | Update existing doc by ID
471 vector_store.delete() | Delete docs by ID list
472
473 -----
474 IMPORTANT NOTES FOR PRODUCTION
475 -----
476
477 1. Use persist_directory for production so data is saved to disk.
478    Without it, data is lost when app restarts.
479
480 2. Generate embeddings ONCE and store them. Do not regenerate
481    every time – it's expensive (API calls + time).
482
483 3. When switching vector stores (Chroma → Pinecone), only
484    change the import + initialization. All method calls remain
485    the same in LangChain.
486
487 4. Metadata filtering is very powerful in RAG apps –
488    use it to narrow search results before semantic ranking.
489
490 5. For prototyping / learning: use ChromaDB (lightweight, local).
491    For production with large data: use Pinecone or Qdrant.
492
493 -----
494 NEXT TOPIC (Preview)
495 -----
496 The video mentions one more component is left before building a
497 complete RAG application. The full RAG pipeline will be built
498 in the next video after covering that final component.
499
500 =====
501                END OF NOTES
502 =====
503
```